L08. Combinational Logic Technologies

Lecture 08

8.1 기본 논리 구성요소 (교재 4.2)

> **재료는 세 가지: fixed gates, look-up table, template-based logic.**

Fixed gates

> **기능이 고정돼 있어 프로그램 불가능한 게이트.**

Look-up table (LUT)

> **입력값으로 표를 인덱싱해 함수 출력을 찾아봄.**

종류	ROM (read only memory), MUX (multiplexer)
구현 순서	먼저 입력 조합별 출력을 적은 진리표(LUT)를 만들고, 그 진리표를 그대로 하드웨어(ROM 또는 MUX)에 옮겨 구현.



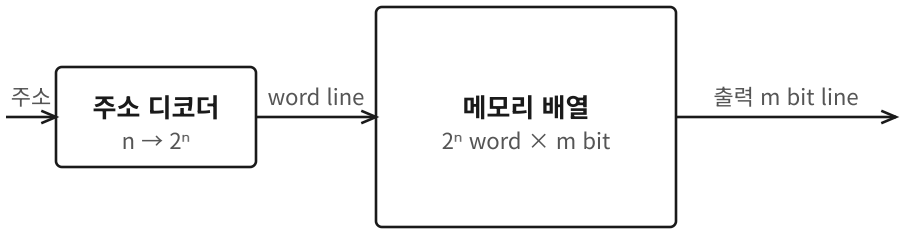
LUT: 입력이 표의 주소 역할을 하고 그 주소에 저장된 값이 출력이 된다.

ROM (Read Only Memory)

> **읽기 전용 메모리. 구조는 디코더 + 메모리 배열.**

동작	Mask ROM은 제조할 때 데이터를 기록하며 읽기만 가능. PROM·EPROM·Flash 등은 제조 후에도 기록할 수 있음.

비휘발성	non-volatile 이라 전원을 꺼도 데이터가 남음.
word line	n입력 주소 디코더가 2^n개 word line 중 하나를 활성화.
bit line	ROM 출력에 해당하는 세로 선 하나하나.



ROM 의 구조: 주소가 디코더로 word line 하나를 켜고 메모리 배열에 저장된 m 비트가 bit line 으로 나온다.

8-word × 4-bit ROM (주소 3비트, 출력 F_0~F_3)

A	B	C	F_0	F_1	F_2	F_3
0	0	0	1	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	1	1
0	1	1	0	1	0	1
1	0	0	1	1	0	0
1	0	1	0	0	0	1
1	1	0	1	1	1	1
1	1	1	0	1	0	1

주소 ABC 에 해당하는 word line 하나가 켜지고, 거기 저장된 4비트가 F_0~F_3 로 읽힘.

Multiplexer (MUX)

- MUX 는 입력 하나를 골라 출력으로 넘김.
- 고르는 건 selection signal(control input).
- 2^n:1 MUX 는 선택 신호 n개로 입력 2^n개 중 하나를 출력 Z 에 전기적 또는 논리적으로 이어 줌.

- 선택 신호가 3비트(A, B, C)면 입력 몇 개 중에서 하나를 고르나? $2^3 = 8$개.

2:1 MUX

I_1	I_0	A	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$Z = A'I_0 + AI_1$. A = 0 이면 $Z = I_0$, A = 1 이면 $Z = I_1$ (functional form, logical form).

MUX 식

2:1 $Z = A'I_0 + AI_1$

4:1 $Z = A'B'I_0 + A'BI_1 + AB'I_2 + ABI_3$

8:1 $Z = A'B'C'I_0 + A'B'CI_1 + A'BC'I_2 + A'BCI_3 + AB'C'I_4 + AB'CI_5 + ABC'I_6 + ABCI_7$

구현 예. 2:1 MUX 는 AND-OR, 4:1 MUX 는 AND-OR 또는 NAND-NAND.

8.2 MUX 로 조합 논리 설계하기

- 하드웨어 look-up table: n비트 선택 입력의 MUX 는 n변수 진리표를 하드웨어로 바로 구현함.
- 선택 신호가 n개보다 적은 MUX 로도 n변수 진리표를 구현할 수 있음.
- n변수 부울함수는 선택 신호가 k개인 2^k:1 MUX로 구현할 수 있다.
- 이때 k개의 변수를 선택 신호로 연결하고, 각 데이터 입력에는 해당 선택 신호 조합에서 원래 함수를 평가한 나머지 n−k개 변수의 함수(restricted function)를 연결한다.
- 남은 변수가 하나뿐이면 각 데이터 입력은 0, 1, 그 변수, 또는 그 변수의 보수 중 하나가 되고, 남은 변수가 둘이상이면 데이터 입력에 별도 논리 게이트로 만든 복합 함수가 필요할 수 있다.

예 · $F(A,B,C) = m_0 + m_2 + m_6 + m_7 = A'B'C' + A'BC' + ABC' + ABC$

8:1 MUX 의 선택 신호 S2 S1 S0 에 A B C(LSB)를 연결하고, 데이터 입력 I0~I7 에는 진리표의 F 값(0 또는 1)을 그대로 연결.

A	B	C	F	입력
0	0	0	1	I0 = 1
0	0	1	0	I1 = 0
0	1	0	1	I2 = 1
0	1	1	0	I3 = 0
1	0	0	0	I4 = 0
1	0	1	0	I5 = 0
1	1	0	1	I6 = 1
1	1	1	1	I7 = 1

같은 F 를 4:1 MUX 로 구현: 선택 신호에 A, B 를 연결하고 나머지 변수 C 는 데이터 입력으로 처리.

부울식으로 구하기

부울식으로 4:1 MUX의 데이터 입력 구하기 — 주어진 함수는 F(A,B,C) = Σm(0,2,6,7)이며, 선택 신호를 S_1=A, S_0=B로 정한다.

① Minterm 전개: F = A′B′C′ + A′BC′ + ABC′ + ABC.

② 공통항 정리: ABC′ + ABC = AB(C′ + C) = AB. 따라서 F = A′B′C′ + A′BC′ + AB.

③ 선택 신호 조합별로 분리: F = A′B′(C′) + A′B(C′) + AB′(0) + AB(1). 여기서 AB′(0)은 항상 0이므로 추가해도 함수가 변하지 않는다.

④ 4:1 MUX의 일반식과 비교: Z = A′B′I_0 + A′BI_1 + AB′I_2 + ABI_3. 대응하는 데이터 입력은 I_0=C′, I_1=C′, I_2=0, I_3=1이다.

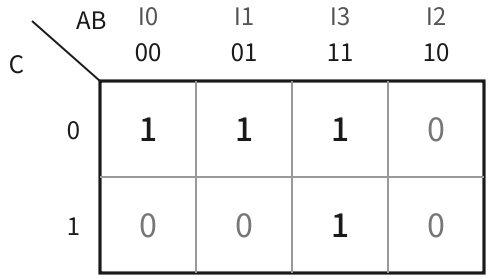
진리표로 구하기

선택 신호 A, B 의 값이 같은 행끼리 묶고, 묶음마다 F 가 C 의 어떤 함수인지 읽음.

A	B	C	F	데이터 입력
0	0	0	1	I_0 = C′
0	0	1	0	
0	1	0	1	I_1 = C′
0	1	1	0	
1	0	0	0	I_2 = 0
1	0	1	0	
1	1	0	1	I_3 = 1
1	1	1	1	

K-map 으로 구하기

C 를 행, AB 를 열로 둔 맵. 열 하나가 MUX 의 데이터 입력 하나(열 순서대로 I_0, I_1, I_3, I_2). 각 열의 두 칸을 C 의 함수로 읽음.



열 00: (1, 0) → C′. 열 01: (1, 0) → C′. 열 11: (1, 1) → 1. 열 10: (0, 0) → 0.

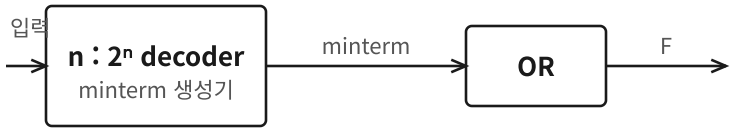
세 방법 모두 $I_0 = C'$, $I_1 = C'$, $I_2 = 0$, $I_3 = 1$.

8.3 Template-based logic

- 미리 정해진 구조에서 연결을 정해 원하는 함수를 구현하는 설계 틀. 예: decoder/demultiplexer, PLA(programmable logic array).

Decoder / DeMUX

DeMUX	입력 하나를 여러 출력 중 하나로 보냄. n비트 제어 입력이 2^n개 출력 중 하나를 선택.
Decoder	코드화된 입력을 코드화된 출력으로 바꿈. minterm generator 로 쓰임. N비트 입력, 2^N비트 출력이면 N:2^N decoder.
크기	1:2 decoder = 1비트 minterm 생성기, 2:4 decoder = 2비트 minterm 생성기.
논리 구성요소	n변수 SoP 식은 어느 것이든 n:2^n decoder + OR 게이트로 구현 가능.



Decoder 의 출력 minterm 중 필요한 것만 OR 로 합치면 SoP 식이 된다.

예 · 4:16 decoder (Enable 있음) 로 3개 함수 구현

이 예에서는 active-high 출력을 가진 4:16 decoder를 사용하고, Enable을 항상 활성 상태로 유지한다고 가정한다. Enable이 active-high이면 EN=1로 연결한다.

$F_1 = A'BC'D + A'B'CD + ABCD = \Sigma m(3, 5, 15) = Y_3 + Y_5 + Y_{15}$

$F_2 = ABC'D' + ABCD' + ABCD = \Sigma m(12, 14, 15) = Y_{12} + Y_{14} + Y_{15}$

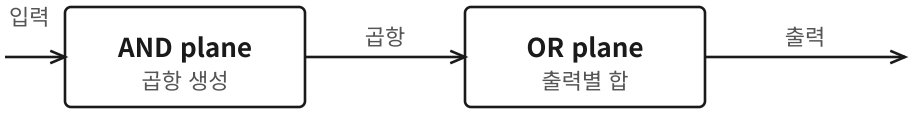
$F_3 = (A' + B' + C' + D') = (ABCD)' = Y_{15}'$, decoder 출력 15 를 인버터로 반전

각 함수는 해당 minterm 의 decoder 출력을 OR 게이트로 합쳐서 구현. F_3 은 minterm 하나(m_{15})짜리 함수의 보수라서 OR 없이 반전만 하면 됨.

PLA (Programmable Logic Array)

> **AND 게이트 배열(AND plane)과 OR 게이트 배열(OR plane)로 이루어진 미리 설계된 빌딩 블록.**

개별화	제조 시 또는 현장에서(in the field) 게이트 연결을 만들거나 끊어서 개별화.
AND plane	입력과 그 보수가 지나는 선 위에서 필요한 곱항을 프로그래밍.
OR plane	어느 곱항을 어느 출력에 더할지 프로그래밍.
특징	ROM은 전체 입력 조합을 디코딩. PLA는 필요한 곱항만 만들고 출력끼리 공유하므로, 함수에 따라 ROM보다 작게 구현할 수 있음.



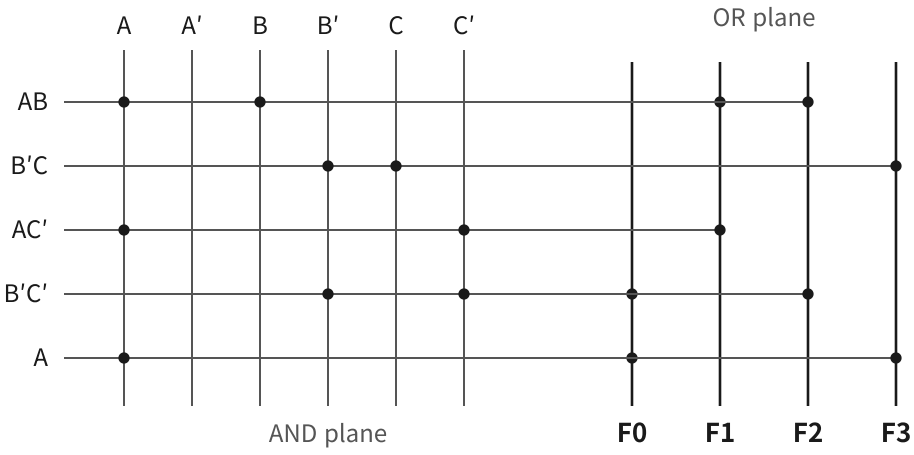
PLA: AND plane 에서 곱항을 만들고 OR plane 에서 출력별로 더한다.

PLA 구현 예

$F_0 = A + B'C'$, $F_1 = AC' + AB$, $F_2 = B'C' + AB$, $F_3 = B'C + A$

곱항	A	B	C	F_0	F_1	F_2	F_3
AB	1	1	–	0	1	1	0
B′C	–	0	1	0	0	0	1
AC′	1	–	0	0	1	0	0
B′C′	–	0	0	1	0	1	0
A	1	–	–	1	0	0	1

PLA personality matrix 읽는 법 — PLA personality matrix는 AND plane에서 생성할 곱항과 OR plane에 연결할 곱항을 정의한 프로그래밍 표다. A, B, C 입력 열에서 1은 해당 변수의 비반전 리터럴을 연결하고, 0은 반전 리터럴을 연결하고, –는 그 변수를 곱항에 사용하지 않음을 뜻한다. F_0~F_3 출력 열에서 1은 해당 곱항을 그 출력의 OR에 연결하고, 0은 연결하지 않음을 뜻한다. 예를 들어 AB 행의 입력 패턴 (1,1,–)는 곱항 AB를 생성한다는 뜻이며, 출력 패턴 (0,1,1,0)은 AB를 F_1과 F_2에 연결한다는 뜻이다. 마찬가지로 B′C′는 F_0과 F_2가 공유하고, A는 F_0과 F_3가 공유한다.

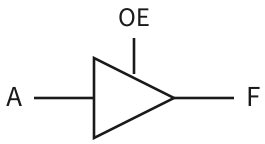


프로그래밍한 PLA(personality matrix). 곱항 AB, B′C′, A 는 여러 출력이 공유한다.

8.4 Tri-state 출력 (교재 4.4)

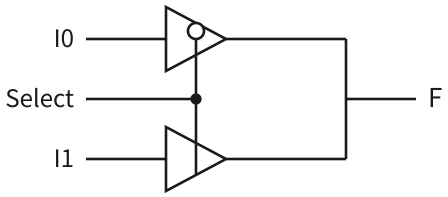
- **Buffer**(OUT = IN): 게이트 출력의 구동 능력을 키울 때 씀(bubble 없음).

- **Tri-state**: 구동 능력과 관련된 세 번째 상태 “Z”. 논리 0, 1 에 더해 Z 상태를 가진 게이트.
- Z 는 출력 임피던스가 매우 높은 상태(이상적으로 무한대). 곧 연결이 없는 것(no connection).



tri-state buffer. OE = 1 이면 F = A, OE = 0 이면 F 는 끊긴다(Z).

A	OE	F
X	0	Z
0	1	0
1	1	1



Tri-state buffer 두 개로 구현한 2:1 MUX. 두 buffer의 출력은 공통 출력선 F에 연결한다. 위쪽 buffer의 데이터 입력은 I_0, 아래쪽 buffer의 데이터 입력은 I_1이다. 위쪽 OE에는 Select의 보수가, 아래쪽 OE에는 Select가 그대로 연결되므로 Select=0이면 F=I_0, Select=1이면 F=I_1이 되어 앞 절의 2:1 MUX 정의 $Z = A'I_0 + AI_1$과 일치한다.